

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

Technical Requirement ID: TR-SQG-001

Related Functional Requirement(s): FR-SQG-001: Generate SQL Query for Sales Orders Data

Objective: Implement a system to generate a complete SQL SELECT statement that applies transformation logic, includes appropriate joins, and ensures correct data types and formatting based on the provided tabular mapping document.

Target Cluster Configuration:

Cluster Name: Data Integration Cluster

Databricks Runtime Version: 7.3 LTS

Node Type: Standard_DS3_v2

Driver Node: Standard_DS3_v2

Worker Nodes: 2-4 (autoscaling)

Autoscaling: Enabled

Auto Termination: Enabled (after 30 minutes of inactivity)

Libraries Installed: Spark SQL, Scala, Python

Source Data Details:

Dataset Name: Mapping Document

Location: Database table (e.g., `mapping.sales\_orders\_comp\_stg`)

Format: Relational table

Description: Contains the mapping rules for generating the SQL query

Dataset Name: Source Tables (e.g., `b\_source.ord\_hdr`, `b\_source.ord\_dtl`, `s\_master.sale\_orders`)

Location: Database tables

Format: Relational tables

Description: Contain the source data for generating the SQL query

Target Data Details:

Output Name: Generated SQL Query

Location: Output file (e.g., `/output/sql\_query.sql`)

Format: Text file

Description: Contains the generated SQL query

Job Flow / Pipeline Stages:

- Retrieve the mapping document for sales_orders_comp_stg
- Generate a SQL SELECT statement for sales_orders_comp_stg
- Retrieve the mapping document for sales_orders_comp
- Generate a SQL SELECT statement for sales_orders_comp
- Format the generated SQL query for readability

Data Transformations / Business Logic:

Step: Apply transformation logic as specified in the mapping document

Description: Use Scala or Python to apply the transformation logic

Transformation Logic:

- Joining tables based on specified join conditions
- Applying transformation logic (e.g., CONCAT, date conversions)
- Using subqueries and derived tables as necessary
- Handling null values and default values as specified in the mapping document

Step: Handle conditional logic (e.g., IF, CASE)

Description: Use Scala or Python to handle conditional logic

Transformation Logic: Apply IF, CASE, and other conditional logic as described in the mapping document

Error Handling and Logging:

- Log errors and exceptions to a log file (e.g., `/logs/error.log`)
- Handle null values and default values as specified in the mapping document
- Provide meaningful error messages for debugging purposes